JavaScript 1.10

Promises & async/await

Print-first interview refresher | outcome → dependency → ownership

Ask two questions:
who observes each failure?
who may apply this result?

1 | A promise represents one outcome, not a lazy operation

Pending	Fulfilled	Rejected
result not settled yet	successful final outcome	failed final outcome

pending → fulfilled OR rejected. Settlement is final; observers do not restart the operation.

Resolved may still be pending: resolution can lock in another promise before it settles.

2 | Every then creates a new promise: follow the returned outcome

return value	no return	throw reason	Promise / thenable
fulfill with value	fulfill with undefined	reject with reason	adopt its outcome

Return every dependency. A catch on the original promise does not protect all new branches.

finally waits for returned cleanup; it preserves the old outcome unless cleanup fails.

3 | Choose the aggregation contract

Method	Rule	Empty input
all	all successes; reject on the first failure	fulfills []
allSettled	wait for a report of every outcome	fulfills []
race	first settlement: success OR failure	stays pending
any	first success; otherwise AggregateError	rejects AggregateError

Combinators do not call input functions or cancel losing operations. Observe all started branches.

4 | Real dependency or accidental waterfall?

Profile	→	Documents(profile.id)	Settings
start when needed		requires profile first	independent: start early

await pauses this async function, not the thread. Awaiting an array does not await its elements.

5 | Cancellation is an operation contract

Owner signal	→	Supporting API	→	Cleanup
one-shot; preserve reason		stop work / reject its result		timer and listener ownership

Abort does not roll back server effects. race with a timeout does not cancel the request.

Fetch Response ≠ HTTP success ≠ parsed body. Aborting can also fail an unfinished body read.

6 | Latest request wins: abort AND verify the version

New request → increment version → replace controller → abort previous work.

Apply result or error only if: owner alive AND version current AND signal not aborted.

Old finally must not clear a new controller. Dispose invalidates pending results and removes the UI.

Test B before A, late failure, current failure, close-before-response and an abort-ignoring source.

Trace outcomes before optimizing syntax

Executor is synchronous. return await can matter for local catch/finally. void is not an error handler.